AutoSRE: ML implementation report

Root-cause analysis for a banking chain: Frontend, API Gateway, Transaction Service, Database. Code: github.com/vaishnavi-nss/autosre-devops (branch ml-cloud-additions).

1. What the ML part is

- **Rule-based correlation engine (the decision maker).** Prometheus alerts are grouped by Alertmanager and posted to the engine. The deepest alerting service in the dependency chain is chosen, a signature table gives the root cause, and a weighted confidence is attached. Unknown patterns escalate to a human instead of guessing.
- **Learned scorer (a second opinion, not the decision maker).** A separate read-only service pulls raw metrics from Prometheus, scores each service, and says agree / disagree / no opinion on the engine's diagnosis. It runs in shadow mode.
- **Evaluation.** A replay baseline and the learned model were compared on a public benchmark, then both were tested on a live cluster.

2. How the learned scorer works

- **Features:** per service, a robust z-score of how far each metric (cpu, memory, latency, errors, workload) moved from its own earlier behaviour.
- **Models:** a random forest ranks services as the culprit; a calibrated gradient-boosting classifier names the fault. Confidence is P(service) x P(fault), Platt-scaled on out-of-fold predictions.
- **Data:** RCAEval (MIT licence), RE1 375 cases and RE2 270 cases, on Online Boutique, Sock Shop and Train Ticket. Evaluation is leave-one-system-out, so the model is always tested on a system it never saw.

3. Results on the public benchmark (RCAEval)

Suite	Replay baseline (rule-style)	Learned model
RE1 (375 cases)	28% wrong, 37% right	61% right when forced to act; 16% wrong when gated at 0.5
RE2 (270 cases)	48% wrong	63.7% right; precision 69% at gate 0.5 up to 82% at 0.9

- Remaining errors are inherent confusions: cpu vs disk, delay vs loss, memory read as disk. Adding logs gave no gain.

4. Results on our live AWS cluster

Faults were injected into a running cluster; the alert pipeline and engine were real.

Measure	Result
Rule engine, 32 live trials	0 wrong actions
Scorer vs the engine's 13 correct decisions	agreed 3, disagreed 8, no opinion 2
Scorer on true diagnoses (26 probes)	agreed 9, disagreed 15
Scorer on deliberately wrong diagnoses (30 probes)	agreed 0, disagreed 27
Scorer picks the right service	62% (91-95% on the benchmark)

- **Honest conclusion:** the benchmark-trained scorer did not transfer to our system. It read latency-led faults as packet loss. It is kept in shadow mode; gate mode would have blocked 8 of 13 correct fixes.

5. Cloud work done (AWS)

- **Environment:** AWS Academy Learner Lab, one t3.large (larger types are denied), minikube single-node cluster, kube-prometheus-stack (Prometheus, Alertmanager, Grafana), LabInstanceProfile for S3 access (no IAM roles can be created).

- **One-command bootstrap:** scripts/aws/bootstrap.sh installs docker, kubectl, minikube and helm, builds the service images inside minikube, deploys AutoSRE and the monitoring stack, installs the Grafana dashboard, runs the live trials, and uploads reports to an S3 bucket.
- **Grafana dashboard:** latency, error ratio, pod restarts, pool utilisation and the engine's confidence, verified against the live cluster.
- **Fault injection:** chaos endpoints (pool, disk, memory, latency, errors) behind CHAOS_ENABLED, a load generator, and a harness that injects, waits for the real alert pipeline, and records every incident with ground truth.
- **Two runs:** run 1 validated the rule engine; run 2 added the second-opinion image (built on the instance) and probed it. 32 trials in total, 0 wrong actions.
- **Bug found only live:** the engine image did not copy second_opinion.py and crash-looped. Fixed, with a test that checks every imported module is in each image.
- **Housekeeping:** instances terminated after each run; the S3 bucket is kept with the raw results. Dispatcher cooldown should be at least 10 minutes because alerts stay firing about 5 minutes after a fault ends.

6. What is built next

- A scorer trained on our own injected faults: a fast collection mode (about 5 min a trial), new disk and connection-pool signals, and a trainer with leave-one-trial-out evaluation. Tested on synthetic data only; no accuracy figure yet.
- It needs one more AWS run (about 2.5 hours of lab time) before any claim about its accuracy.

7. Where to find it

- **correlation-engine/**: rules, engine, evaluation scripts (rcaeval_eval.py, live_validate.py).
- **second-opinion/**: the scorer service, train.py (benchmark), train_own.py (our data).
- **scripts/aws/**, **k8s/**, **monitoring/**, **docs/aws-validation.md**: cloud setup and how to repeat the run.
- **docs/evaluation.md**, **docs/rcaeval-results.md**, **docs/live-validation-run2/report.md**: full numbers.